

NEURAL NETWORKS

Foundations: Neural Networks to Representation Learning

Author: Eng. Carlos Andrés Sierra, M.Sc.
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

2026-III



UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

- 1 Neural Network Fundamentals
- 2 Optimization & Regularization



Outline

^

1 Neural Network Fundamentals

2 Optimization & Regularization

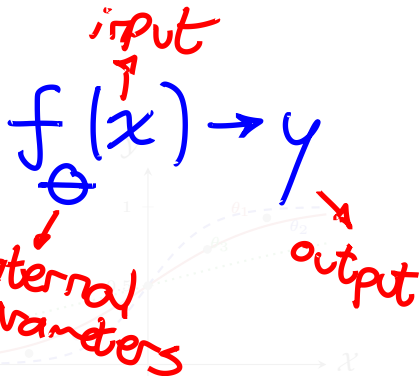


What is a Neural Network?

Definition

A neural network is a parametric function $f_{\theta} : \mathcal{X} \rightarrow \mathcal{Y}$ composed of simple, interconnected computational units (neurons).

- **Neuron:** weighted sum + nonlinear activation
- **Layer:** neurons operating in parallel
matrix: x
↓
linear equations
- **Depth:** key enabler of hierarchical learning
- **Training:** adjusting θ so that $f_{\theta}(x) \approx y$



What is a Neural Network?

Definition

A neural network is a **parametric function** $f_{\theta} : \mathcal{X} \rightarrow \mathcal{Y}$ composed of simple, interconnected computational units (*neurons*).

- **Neuron:** weighted sum + nonlinear activation

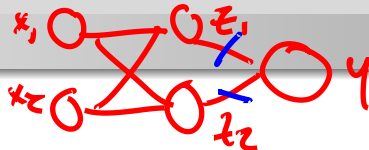
$$- w_3 x_1 + w_4 x_2 = z_1$$

$$w_3 x_1 + w_4 x_2 = z_2$$

$$y = w_5 z_1 + w_6 z_2$$

$$= w_5 w_3 x_1 + w_5 w_4 x_2 + w_6 w_3 x_1 + w_6 w_4 x_2$$

$$= (w_5 w_3 + w_6 w_3) x_1 + (w_5 w_4 + w_6 w_4) x_2$$



$$z_0 = \underbrace{w_1 x_1 + w_2 x_2 + \dots}_{\text{weights}}$$

↓

more inference

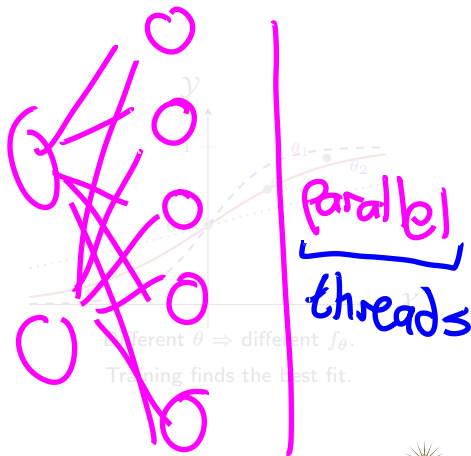


What is a Neural Network?

Definition

A neural network is a **parametric function** $f_{\theta} : \mathcal{X} \rightarrow \mathcal{Y}$ composed of simple, interconnected computational units (*neurons*).

- **Neuron:** weighted sum + nonlinear activation
- **Layer:** neurons operating in parallel
- **Depth:** key enabler of hierarchical features
- **Training:** adjust θ so $f_{\theta}(x) \approx y$



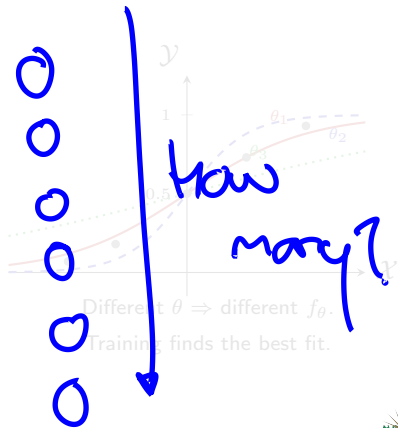
What is a Neural Network?

Definition

A neural network is a **parametric function** $f_{\theta} : \mathcal{X} \rightarrow \mathcal{Y}$ composed of simple, interconnected computational units (*neurons*).

- **Neuron:** **weighted sum** + nonlinear activation
- **Layer:** neurons operating in parallel
- **Depth:** key enabler of **hierarchical features**
- **Training:** adjust θ so $f_{\theta}(x) \approx y$

How many layers?



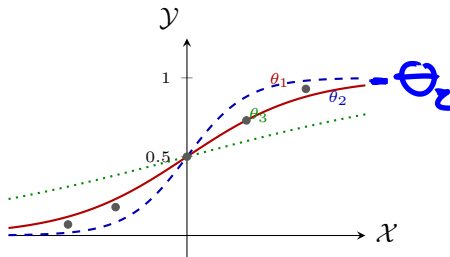
What is a Neural Network?

Definition

A neural network is a **parametric function** $f_{\theta} : \mathcal{X} \rightarrow \mathcal{Y}$ composed of simple, interconnected computational units (*neurons*).

- **Neuron:** **weighted sum** + nonlinear activation
- **Layer:** neurons operating in parallel
- **Depth:** key enabler of **hierarchical features**
- **Training:** adjust θ so

$$f_{\theta}(x) \approx y$$



Different $\theta \Rightarrow$ different f_{θ} .

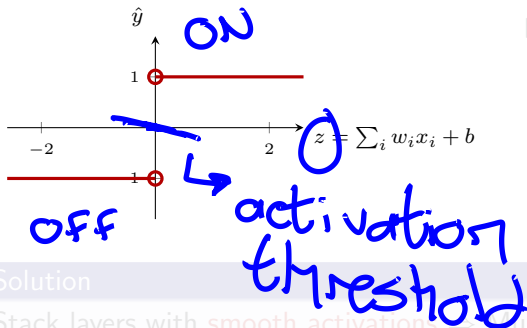
Training finds the best fit.



The Perceptron

Rosenblatt (1958)

$$\hat{y} = \text{sign}(\sum_i w_i x_i + b)$$



Strengths

- Binary classification
- Convergence guarantee (linearly separable data)

Limitations

- Cannot solve XOR
- Only linearly separable
- Hard threshold — not differentiable

Solution

Stack layers with smooth activations → Multi-Layer Perceptron

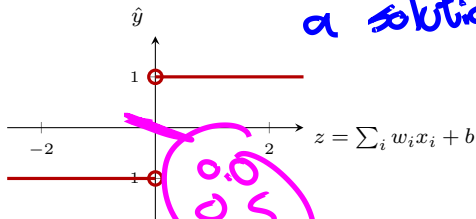


The Perceptron

1957 - AI. *Handwritten: Rosenblatt (1958)*

$$\hat{y} = \text{sign}(\sum_i w_i x_i + b)$$

Handwritten: there is a solution



Strengths

- Binary classification
- Convergence guarantee (*linearly separable data*)

Limitations

- Cannot solve XOR
- Only linearly separable
- Hard threshold — not differentiable

Solution

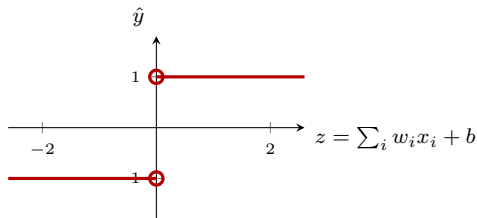
Stack layers with smooth activations $\Rightarrow$ Multi-Layer Perceptron



The Perceptron

Rosenblatt (1958)

$$\hat{y} = \text{sign}(\sum_i w_i x_i + b)$$



Strengths

- Binary classification
- Convergence guarantee (*linearly separable data*)

Limitations

- Cannot solve **XOR**
- Only linearly separable
- Hard threshold — not differentiable

Solution

Stack layers with smooth activations $\Rightarrow$ Multi-Layer Perceptron



Multi-Layer Perceptron (MLP) — Architecture

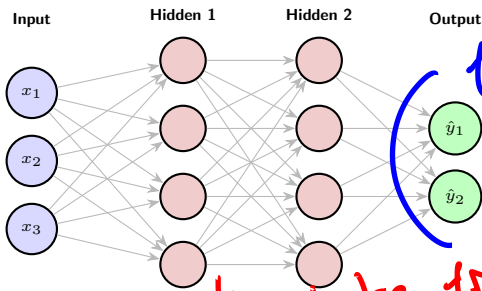
Architecture

Input layer → Hidden layer(s) → Output layer

For each layer l :

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)})$$

DAG
↓
Directed
Acyclic
Graph



threshold
"new idea"

knowledge transfer



Multi-Layer Perceptron (MLP) — Architecture

Architecture

Input layer → Hidden layer(s) → Output layer

- **Universal Approximation** (*Hornik, 1991*): one hidden layer with enough neurons approximates **any** continuous function

• Depth more parameter-efficient than extreme width



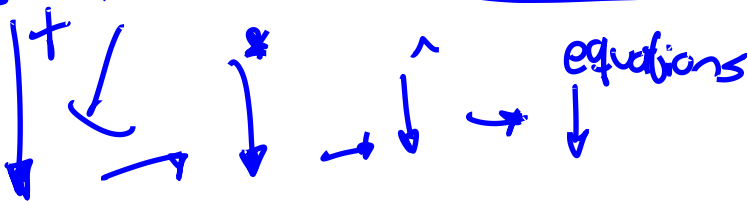
Multi-Layer Perceptron (MLP) — Architecture

Architecture

Input layer $\rightarrow$ Hidden layer(s) $\rightarrow$ Output layer

- **Universal Approximation** (*Hornik, 1991*): one hidden layer with enough neurons approximates **any** continuous function

- **Depth** is more parameter-efficient than extreme width

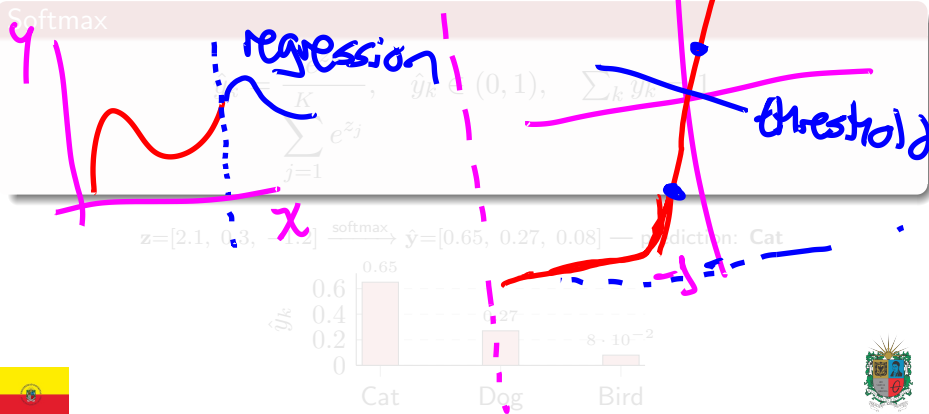


Multi-Layer Perceptron (MLP) — Output & Softmax

Output layer choices

- **Linear:** regression
- **Sigmoid:** binary classification
- **Softmax:** multi-class $\rightarrow$ probability distribution over K classes

Softmax



Multi-Layer Perceptron (MLP) — Output & Softmax

Output layer choices

- **Linear:** regression
- **Sigmoid:** binary classification
- **Softmax:** multi-class → probability distribution over K classes

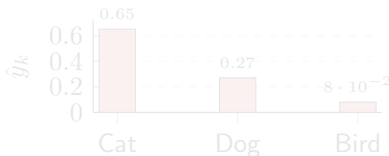
Handwritten notes: **Fuzzy logic**

Softmax

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad \hat{y}_k \in (0, 1), \quad \sum_k \hat{y}_k = 1$$

Handwritten note: **convex combination**

$$\mathbf{z} = [2.1, 0.3, -1.2] \xrightarrow{\text{softmax}} \hat{\mathbf{y}} = [0.65, 0.27, 8 \cdot 10^{-2}]$$



Multi-Layer Perceptron (MLP) — Output & Softmax

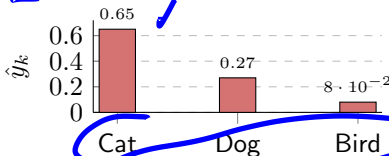
Output layer choices

- **Linear**: regression **Sigmoid**: binary classification
- **Softmax**: multi-class $\rightarrow$ probability distribution over K classes

Softmax

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad \hat{y}_k \in (0, 1), \quad \sum_k \hat{y}_k = 1$$

$z = [2.1, 0.3, -1.2] \xrightarrow{\text{softmax}} \hat{y} = [0.65, 0.27, 0.08] \text{ — prediction: Cat}$



K categories



Activation Functions — no-linear!

Sigmoid:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \in (0, 1)$$

Tanh:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \in (-1, 1)$$

ReLU: $\max(0, x)$ — default
today

hidden

GELU: $x \cdot \Phi(x)$ — used in
Transformers

LLMs



Why ReLU dominates

- No saturation for positive inputs
- Sparse activation (many zeros)
- Computationally trivial
- Mitigates vanishing gradients

$$\hat{y} = (\sim)x_1 + H(\sim)x_2$$



Activation Functions

Sigmoid:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \in (0, 1)$$

Tanh:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \in (-1, 1)$$

ReLU: $\max(0, x)$ — **default today**

GELU: $x \cdot \Phi(x)$ — used in Transformers

Why ReLU dominates

- No saturation for positive inputs
- Sparse activation (many zeros)
- Computationally trivial
- Mitigates vanishing gradient

$(Ax+b)$

$$\frac{1}{1 + e^x}$$

hidden layer
neuron output



Activation Functions — Visual Overview

binary

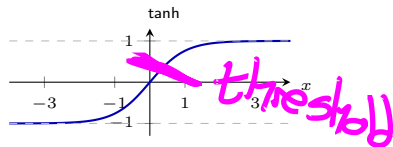
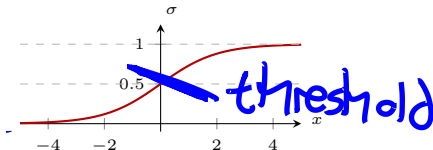
Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

output

Tanh

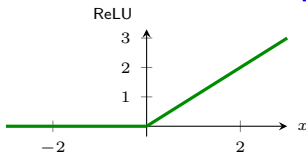
$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



ReLU

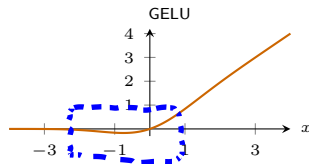
$$\max(0, x)$$

hidden



GELU

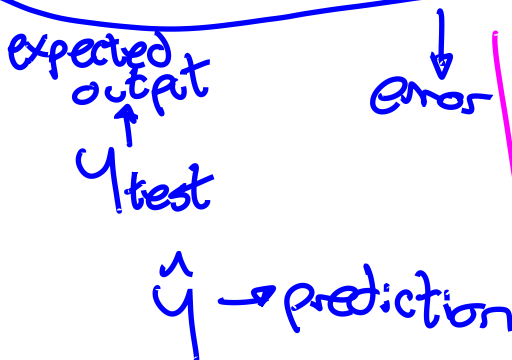
$$x \cdot \Phi(x)$$



Loss Functions → error measure

The Objective of Training

Minimise $\mathcal{L}(\theta)$ — a measure of disagreement between predictions and targets over the training set.



Loss Functions

Mean Squared Error (regression)

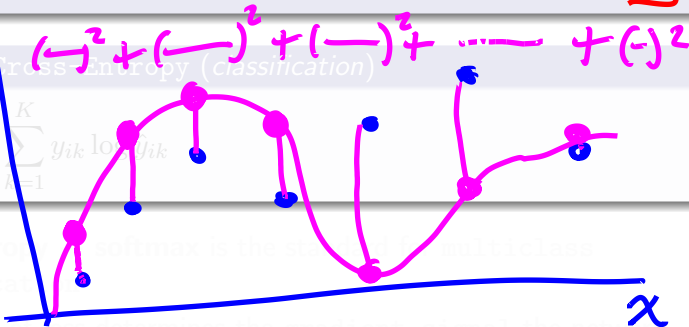
$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N \|\hat{y}_i - y_i\|^2$$

→ forecasting

Categorical Cross-Entropy (classification)

$$\mathcal{L}_{\text{CCE}} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log \hat{y}_{ik}$$

$y_{\text{test}} = \bullet$
 $y_{\text{pred}} = \bullet$



Loss Functions

Mean Squared Error (*regression*)

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N \|\hat{y}_i - y_i\|^2$$

Categorical Cross-Entropy (*classification*)

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log \hat{y}_{ik}$$

Binary

Confusion
Matrix

differentiation



Loss Functions

Mean Squared Error (*regression*)

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N \|\hat{y}_i - y_i\|^2$$

Categorical Cross-Entropy (*classification*)

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log \hat{y}_{ik}$$

- **Cross-entropy + softmax** is the standard for **multiclass classification**
- The choice of loss determines the gradient signal the network receives



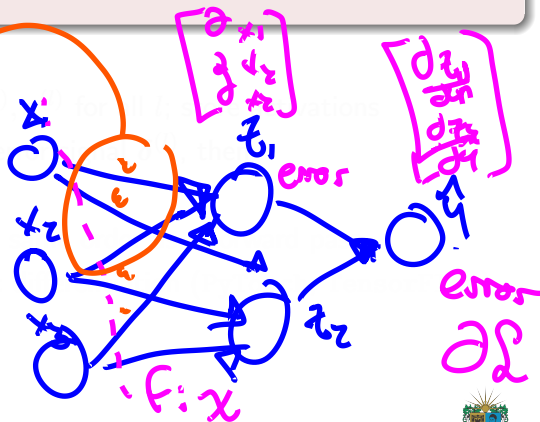
Backpropagation

Core Idea

Apply the **chain rule** repeatedly to compute $\partial \mathcal{L} / \partial \mathbf{W}^{(l)}$ for every layer, from output back to input.

Two-pass algorithm:

1st pass
forward



Backpropagation

Core Idea

Apply the **chain rule** repeatedly to compute $\partial \mathcal{L} / \partial \mathbf{W}^{(l)}$ for every layer, from output back to input.

Two-pass algorithm:

- 1 **Forward pass:** compute $\mathbf{z}^{(l)}, \mathbf{a}^{(l)}$ for all l ; store activations
- 2 **Backward pass:** compute error signal $\delta^{(l)}$, then

$$\nabla_{\mathbf{W}^{(l)}} \mathcal{L} = \sum_j \delta_j^{(l)} \mathbf{a}_j^{(l-1)}$$

Handwritten notes: x_1, x_2 and a circle around f_j with an arrow pointing to $\mathbf{a}^{(l)}$ in the list above.

• Time complexity: $O(n)$ — same order as a forward pass

• Made efficient by automatic differentiation (PyTorch, TensorFlow)



Backpropagation

Core Idea

Apply the **chain rule** repeatedly to compute $\partial \mathcal{L} / \partial \mathbf{W}^{(l)}$ for every layer, from output back to input.

Two-pass algorithm:

- 1 **Forward pass:** compute $\mathbf{z}^{(l)}, \mathbf{a}^{(l)}$ for all l : store activations
- 2 **Backward pass:** compute error signal $\delta^{(l)}$, then

$$\nabla_{\mathbf{W}^{(l)}} \mathcal{L} = \delta^{(l)} (\mathbf{a}^{(l-1)})^\top$$

- Time complexity: $O(|\theta|)$ — same order as a forward pass
- Made efficient by automatic differentiation (PyTorch, TensorFlow)



Backpropagation

Core Idea

Apply the **chain rule** repeatedly to compute $\partial \mathcal{L} / \partial \mathbf{W}^{(l)}$ for every layer, from output back to input.

Two-pass algorithm:

- 1 **Forward pass:** compute $\mathbf{z}^{(l)}, \mathbf{a}^{(l)}$ for all l ; store activations
 - 2 **Backward pass:** compute error signal $\delta^{(l)}$, then

$$\nabla_{\mathbf{W}^{(l)}} \mathcal{L} = \delta^{(l)} (\mathbf{a}^{(l-1)})^\top$$
- Time complexity: $O(|\theta|)$ — same order as a forward pass
 - Made efficient by automatic differentiation (PyTorch, TensorFlow)

$\rightarrow \mathbf{A} + \mathbf{b}$

epoch

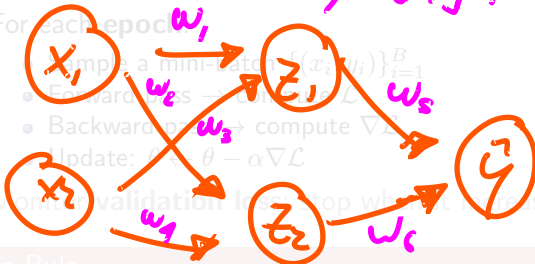


The Training Loop

1 **Initialise** weights θ_0 (random)

2 Split data: train / validation / test

3 For each epoch



4 Sample a mini-batch $\{(x_i, y_i)\}_{i=1}^B$

• Forward pass — compute $\mathcal{L}$

• Backward pass — compute $\nabla \mathcal{L}$

• Update: $\theta \leftarrow \theta - \alpha \nabla \mathcal{L}$

5 Monitor validation loss. Stop when it increases (early stopping)

Golden Rule

Never tune hyperparameters using the test set. The test set measures final generalisation only.

$$\theta_0 = [w_1, w_2, w_3, w_4, b_1, b_2]$$



The Training Loop

- 1 **Initialise** weights θ_0 (random) 0.3
 - 2 Split data: train / validation / test 0.7 0.1 0.2
 - 3 For each epoch:
 - Sample a mini-batch $\{(\mathbf{x}_i, y_i)\}_{i=1}^B$
 - Forward pass $\rightarrow$ compute $\mathcal{L}$
 - Backward pass $\rightarrow$ compute $\nabla \mathcal{L}$
 - Update: $\theta \leftarrow \theta - \eta \nabla \mathcal{L}$
 - 4 Monitor **validation loss**; stop when it increases (early stopping)
- Supervised learning

Golden Rule

Never tune hyperparameters using the test set. The test set measures final generalisation only.

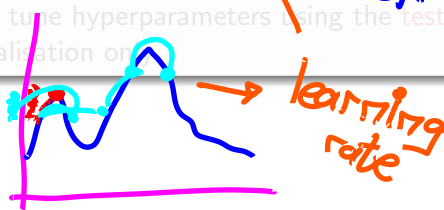


The Training Loop

- 1 **Initialise** weights θ_0 (random)
- 2 Split data: **train** / **validation** / **test**
- 3 For each **epoch**: $\rightarrow \sim \text{iteration}$
 - Sample a mini-batch $\{(x_i, y_i)\}_{i=1}^B$
 - Forward pass $\rightarrow$ compute $\mathcal{L} \rightarrow \text{error (test)}$
 - Backward pass $\rightarrow$ compute $\nabla \mathcal{L} \rightarrow \text{back propagation}$
 - Update: $\theta \leftarrow \theta - \alpha \nabla \mathcal{L}$ $\rightarrow \text{minimize error}$
- 4 Monitor **validation loss**; stop when it increases (early stopping)

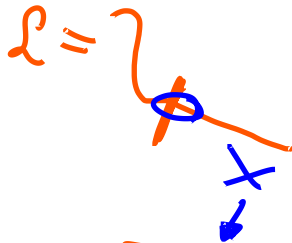
Golden Rule

Never tune hyperparameters using the **test set**. The test set measures final generalisation on



The Training Loop

- 1 **Initialise** weights θ_0 (random)
- 2 Split data: **train** / **validation** / **test**
- 3 For each **epoch**:
 - Sample a mini-batch $\{(x_i, y_i)\}_{i=1}^B$
 - Forward pass $\rightarrow$ compute $\mathcal{L}$
 - Backward pass $\rightarrow$ compute $\nabla \mathcal{L}$
 - Update: $\theta \leftarrow \theta - \alpha \nabla \mathcal{L}$
- 4 Monitor validation loss; stop when it increases (early stopping)



Golden Rule

Never tune hyperparameters using the **test set**. The test set measures final generalisation only.

convergence



The Training Loop

- 1 **Initialise** weights θ_0 (random)
- 2 Split data: **train** / **validation** / **test**
- 3 For each **epoch**:
 - Sample a **mini-batch** $\{(x_i, y_i)\}_{i=1}^B$
 - Forward pass $\rightarrow$ compute $\mathcal{L}$
 - Backward pass $\rightarrow$ compute $\nabla \mathcal{L}$
 - Update: $\theta \leftarrow \theta - \alpha \nabla \mathcal{L}$
- 4 Monitor **validation loss**: stop when it increases (early stopping)

Bayesian
scikit-learn

Golden Rule

Never **tune hyperparameters** using the test set. The test set measures final generalisation only.

learning rate
early stopping

epochs



Practical Example: MLP on Two-Moons

- **Problem:** binary classification on non-linear boundary
- **Architecture:** $2 \rightarrow 32 \rightarrow 32 \rightarrow 1$, ReLU, sigmoid output
- **Experiment:** vary number of hidden layers and width
- **Observation:** deep narrow network generalises better than wide shallow one
- **Workshop connection:** understanding this experiment is the foundation for all classification tasks in Challenges 2, 5, and 7



Practical Example: MLP on Two-Moons

- **Problem:** binary classification on non-linear boundary
- **Architecture:** $2 \rightarrow 32 \rightarrow 32 \rightarrow 1$, ReLU, sigmoid output
- **Experiment:** vary number of hidden layers and width
- **Observation:** deep narrow network generalises better than wide shallow one
- **Workshop connection:** understanding this experiment is the foundation for all classification tasks in Challenges 2, 5, and 7



Practical Example: MLP on Two-Moons

- **Problem:** binary classification on non-linear boundary
- **Architecture:** $2 \rightarrow 32 \rightarrow 32 \rightarrow 1$, ReLU, sigmoid output
- **Experiment:** vary number of hidden layers and width
- **Observation:** deep narrow network generalises better than wide shallow one
- **Workshop connection:** understanding this experiment is the foundation for all classification tasks in Challenges 2, 5, and 7



Practical Example: MLP on Two-Moons

- **Problem:** binary classification on non-linear boundary
- **Architecture:** $2 \rightarrow 32 \rightarrow 32 \rightarrow 1$, ReLU, sigmoid output
- **Experiment:** vary number of hidden layers and width
- **Observation:** deep narrow network generalises better than wide shallow one
- **Workshop connection:** understanding this experiment is the foundation for all classification tasks in Challenges 2, 5, and 7



Practical Example: MLP on Two-Moons

- **Problem:** binary classification on non-linear boundary
- **Architecture:** $2 \rightarrow 32 \rightarrow 32 \rightarrow 1$, ReLU, sigmoid output
- **Experiment:** vary number of hidden layers and width
- **Observation:** deep narrow network generalises better than wide shallow one
- **Workshop connection:** understanding this experiment is the foundation for all classification tasks in Challenges 2, 5, and 7



Outline

- 1 Neural Network Fundamentals
- 2 Optimization & Regularization



Gradient Descent

Update Rule

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} \mathcal{L}(\theta)$$

Full-batch GD

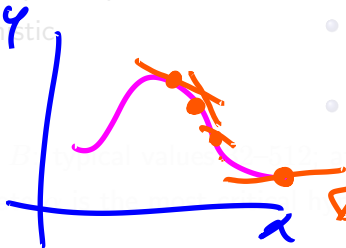
- Exact gradient
- Expensive per step
- Deterministic

Mini-batch SGD

- Noisy gradient estimate
- Cheap per step
- Noise helps escape local minima
- Standard in practice

• Batch size B (typical values: 32–512): affects convergence and memory

• Learning rate is the most important hyperparameter



$\nabla = 0 \rightarrow \text{optimal}$



Gradient Descent

Update Rule

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} \mathcal{L}(\theta)$$

Full-batch GD

- Exact gradient ✓
- Expensive per step
- Deterministic

Mini-batch SGD

- Noisy gradient estimate
- Cheap per step
- Noise helps escape local minima
- Standard in practice

- Batch size B : typical values 32–512; affects convergence and memory
- Learning rate α is the most critical hyperparameter



Gradient Descent

Update Rule

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} \mathcal{L}(\theta)$$

Full-batch GD

- Exact gradient
- Expensive per step
- Deterministic

Mini-batch SGD

- Noisy gradient estimate
- Cheap per step
- Noise helps escape local minima
- **Standard in practice**

- Batch size B : typical values 32–512; affects convergence and memory
- **Learning rate α** is the most critical hyperparameter



Momentum and Nesterov

SGD with Momentum

$$\mathbf{v} \leftarrow \beta \mathbf{v} - \alpha \nabla \mathcal{L}, \quad \theta \leftarrow \theta + \mathbf{v}$$

- Typical $\beta = 0.9$; accumulates gradient history
- Dampens oscillations; accelerates in consistent directions
- Nesterov: compute gradient at the look-ahead position $\theta + \beta \mathbf{v}$ instead of current θ ; slightly better convergence

Intuition

Momentum is like a ball rolling down a hill that builds **inertia**. Nesterov is a ball that **looks ahead** before deciding where to roll.



Momentum and Nesterov

SGD with Momentum

$$\mathbf{v} \leftarrow \beta \mathbf{v} - \alpha \nabla \mathcal{L}, \quad \theta \leftarrow \theta + \mathbf{v}$$

- Typical $\beta = 0.9$; accumulates gradient **history**
- Dampens oscillations; accelerates in consistent directions
- **Nesterov**: compute gradient at the **look-ahead** position $\theta + \beta \mathbf{v}$ instead of current θ — slightly better convergence

Intuition

Momentum is like a ball rolling down a hill that builds **inertia**. Nesterov is a ball that **looks ahead** before deciding where to roll.



Momentum and Nesterov

SGD with Momentum

$$\mathbf{v} \leftarrow \beta \mathbf{v} - \alpha \nabla \mathcal{L}, \quad \theta \leftarrow \theta + \mathbf{v}$$

- Typical $\beta = 0.9$; accumulates gradient **history**
- Dampens oscillations; accelerates in consistent directions
- **Nesterov**: compute gradient at the **look-ahead** position $\theta + \beta \mathbf{v}$ instead of current θ — slightly better convergence

Intuition

Momentum is like a ball rolling down a hill that builds **inertia**. Nesterov is a ball that **looks ahead** before deciding where to roll.



Momentum and Nesterov

SGD with Momentum

$$\mathbf{v} \leftarrow \beta \mathbf{v} - \alpha \nabla \mathcal{L}, \quad \theta \leftarrow \theta + \mathbf{v}$$

- Typical $\beta = 0.9$; accumulates gradient **history**
- Dampens oscillations; accelerates in consistent directions
- **Nesterov**: compute gradient at the **look-ahead** position $\theta + \beta \mathbf{v}$ instead of current θ — slightly better convergence

Intuition

Momentum is like a ball rolling down a hill that builds **inertia**. Nesterov is a ball that **looks ahead** before deciding where to roll.



Momentum and Nesterov

SGD with Momentum

$$\mathbf{v} \leftarrow \beta \mathbf{v} - \alpha \nabla \mathcal{L}, \quad \theta \leftarrow \theta + \mathbf{v}$$

- Typical $\beta = 0.9$; accumulates gradient **history**
- Dampens oscillations; accelerates in consistent directions
- **Nesterov**: compute gradient at the **look-ahead** position $\theta + \beta \mathbf{v}$ instead of current θ — slightly better convergence

Intuition

Momentum is like a ball rolling down a hill that builds **inertia**. Nesterov is a ball that **looks ahead** before deciding where to roll.



Adam Optimizer

Adaptive Moment Estimation (Kingma & Ba, 2015)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\theta \leftarrow \theta - \alpha \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$$

$$\hat{m}_t = m_t / (1 - \beta_1^t)$$

$$\hat{v}_t = v_t / (1 - \beta_2^t)$$

$B_1 + (1 - \beta_1) = 1$
 $\hookrightarrow$ convex combination



Adam Optimizer

Adaptive Moment Estimation (Kingma & Ba, 2015)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$\hat{m}_t = m_t / (1 - \beta_1^t)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{v}_t = v_t / (1 - \beta_2^t)$$

$$\theta \leftarrow \theta - \alpha \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$$

- Defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, $\alpha = 10^{-3}$
- Per-parameter adaptive learning rates
- **AdamW**: Adam + decoupled weight decay — preferred in current practice
- Adam dominates in deep learning; SGD+Momentum preferred in CV research



Adam Optimizer

Adaptive Moment Estimation (Kingma & Ba, 2015)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$\hat{m}_t = m_t / (1 - \beta_1^t)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{v}_t = v_t / (1 - \beta_2^t)$$

$$\theta \leftarrow \theta - \alpha \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)$$

- Defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, $\alpha = 10^{-3}$
- Per-parameter **adaptive learning rates**
- **AdamW**: Adam + decoupled weight decay — preferred in current practice
- Adam dominates in deep learning; SGD+Momentum preferred in CV research



Adam Optimizer

Adaptive Moment Estimation (Kingma & Ba, 2015)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$\hat{m}_t = m_t / (1 - \beta_1^t)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{v}_t = v_t / (1 - \beta_2^t)$$

$$\theta \leftarrow \theta - \alpha \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)$$

- Defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, $\alpha = 10^{-3}$
- Per-parameter **adaptive learning rates**
- **AdamW**: Adam + decoupled weight decay — preferred in current practice

- Adam dominates in deep learning; SGD+Momentum preferred in CV research



Adam Optimizer

Adaptive Moment Estimation (Kingma & Ba, 2015)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$\hat{m}_t = m_t / (1 - \beta_1^t)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{v}_t = v_t / (1 - \beta_2^t)$$

$$\theta \leftarrow \theta - \alpha \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)$$

- Defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, $\alpha = 10^{-3}$
- Per-parameter **adaptive learning rates**
- **AdamW**: Adam + decoupled weight decay — preferred in current practice
- **Adam** dominates in deep learning; SGD+Momentum preferred in CV research

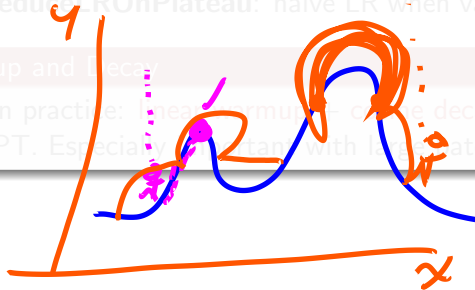


Learning Rate Scheduling

- **Step decay**: multiply LR by factor γ every k epochs
- **Cosine annealing**: $\alpha_t = \alpha_{\min} + \frac{1}{2}(\alpha_{\max} - \alpha_{\min}) \cdot \frac{k}{T} + \cos(\pi t/T)$
- **Linear warmup**: ramp from near-zero for first w steps, then decay
- **ReduceLROnPlateau**: halve LR when validation loss stops improving

Warmup and Decay

Modern practice: linear warmup + cosine decay. Used in ResNets, BERT, and GPT. Especially important with large batch sizes.



LR is a hyperparameter



Learning Rate Scheduling

- **Step decay:** multiply LR by factor γ every k epochs
- **Cosine annealing:** $\alpha_t = \alpha_{\min} + \frac{1}{2}(\alpha_{\max} - \alpha_{\min})(1 + \cos(\pi t/T))$
- **Linear warmup:** ramp from near-zero for first w steps, then decay
- **ReduceLROnPlateau:** halve LR when validation loss stops improving

Warmup and Decay

Modern practice: linear warmup + cosine decay. Used in ResNets, BERT, and GPT. Especially important with large batch sizes.

$$\alpha \nabla \rightarrow f \quad \alpha \rightarrow 0$$

$$\Rightarrow \alpha \nabla \rightarrow 0$$

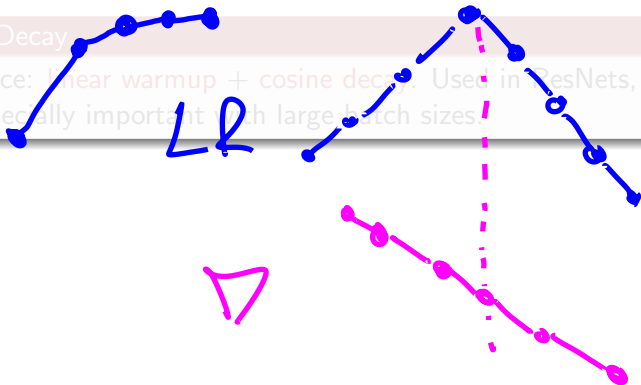


Learning Rate Scheduling

- **Step decay:** multiply LR by factor γ every k epochs
- **Cosine annealing:** $\alpha_t = \alpha_{\min} + \frac{1}{2}(\alpha_{\max} - \alpha_{\min})(1 + \cos(\pi t/T))$
- **Linear warmup:** ramp from near-zero for first w steps, then decay
- ReduceLROnPlateau: halve LR when validation loss stops improving

Warmup and Decay

Modern practice: **linear warmup + cosine decay**. Used in ResNets, BERT, and GPT. Especially important with large batch sizes.



Learning Rate Scheduling

- **Step decay:** multiply LR by factor γ every k epochs
- **Cosine annealing:** $\alpha_t = \alpha_{\min} + \frac{1}{2}(\alpha_{\max} - \alpha_{\min})(1 + \cos(\pi t/T))$
- **Linear warmup:** ramp from near-zero for first w steps, then decay
- **ReduceLROnPlateau:** halve LR when validation loss stops improving

Warmup and Decay

most popular $\rightarrow$ 60%

Modern practice: linear warmup + cosine decay. Used in ResNets, BERT, and GPT. Especially important with large batch sizes.

loss 5... 2 0.8 0.32
| 0.3 0.15 0.07 0.03

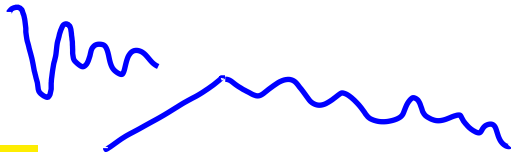


Learning Rate Scheduling

- **Step decay:** multiply LR by factor γ every k epochs
- **Cosine annealing:** $\alpha_t = \alpha_{\min} + \frac{1}{2}(\alpha_{\max} - \alpha_{\min})(1 + \cos(\pi t/T))$
- **Linear warmup:** ramp from near-zero for first w steps, then decay
- **ReduceLROnPlateau:** halve LR when validation loss stops improving

Warmup and Decay

Modern practice: linear warmup + cosine decay. Used in ResNets, BERT, and GPT. Especially important with large batch sizes.



Bias-Variance Trade-Off

$$Ax + b \rightarrow \text{bias}$$

Underfitting (high bias)

- Model too simple
- High training error
- Cannot capture data structure
- Fix: increase capacity

Overfitting (high variance)

- Model too complex
- Low training, high val error
- Memorises noise
- Fix: regularise

Practical Guideline

Start with a model that fits the data, then regularise until train and val errors converge. Never start with a model that underfits.

no pattern detection

eq1

eq2

eq3

eq4



Bias–Variance Trade-Off

Underfitting (high bias)

- Model too simple
- High training error
- Cannot capture data structure
- Fix: increase **capacity**

Overfitting (high variance)

- Model too complex
- Low training, high val error
- Memorises noise
- Fix: **regularise**

Practical Guideline

Start with a model that **slightly overfits**, then **regularise** until train and val errors converge. Never start with a model that underfits.



$$\mathcal{L}_{\text{reg}}(\theta) = \mathcal{L}(\theta) + \Omega(\theta)$$

regularization
penalty

$\Omega(\theta) = \frac{\lambda}{2} \|\mathbf{W}\|_F^2$ drives weights small but rarely zero



L1 and L2 Regularization

Augmented Objective

$$\mathcal{L}_{\text{reg}}(\theta) = \mathcal{L}(\theta) + \Omega(\theta)$$

L2 Regularization (Weight Decay / Ridge)

$$\Omega(\theta) = \frac{\lambda}{2} \|\mathbf{W}\|_F^2 \quad \text{drives weights small but rarely zero}$$

L1 Regularization (Lasso)

$$\Omega(\theta) = \lambda \|\mathbf{W}\|_1 \quad \text{promotes sparse weights (many exact zeros)}$$

penalization factor

• In deep learning (weight decay) is the standard choice

• Elastic Net combines both — used in linear models and sparse NNs



L1 and L2 Regularization

Augmented Objective

$$\mathcal{L}_{\text{reg}}(\theta) = \mathcal{L}(\theta) + \Omega(\theta)$$

L2 Regularization (Weight Decay / Ridge)

$$\Omega(\theta) = \frac{\lambda}{2} \|\mathbf{W}\|_F^2 \quad \text{drives weights small but rarely zero}$$

L1 Regularization (Lasso)

$$\Omega(\theta) = \lambda \|\mathbf{W}\|_1 \quad \text{promotes sparse weights (many exact zeros)}$$

feature importance



L1 and L2 Regularization

Augmented Objective

$$\mathcal{L}_{\text{reg}}(\theta) = \mathcal{L}(\theta) + \Omega(\theta)$$

L2 Regularization (Weight Decay / Ridge)

$$\Omega(\theta) = \frac{\lambda}{2} \|\mathbf{W}\|_F^2 \quad \text{drives weights small but rarely zero}$$

L1 Regularization (Lasso)

$$\Omega(\theta) = \lambda \|\mathbf{W}\|_1 \quad \text{promotes sparse weights (many exact zeros)}$$

- In deep learning: L2 (weight decay) is the standard choice
- Elastic Net: combines both — used in linear models and sparse NNs



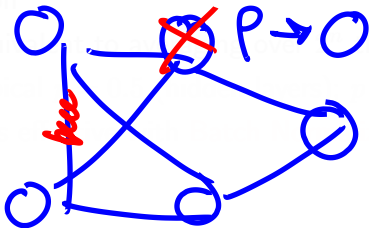
Dropout

Definition (Srivastava et al., 2014)

During training, independently zero out each activation with probability p .
 At inference, multiply activations by $(1 - p)$ (*inverted dropout* scales during training).

• Forces redundant representations — no single neuron can be relied upon

• Equivariant to permutation of hidden units



Dropout

Definition (Srivastava et al., 2014)

During training, independently zero out each activation with probability p . At inference, multiply activations by $(1 - p)$ (*inverted dropout* scales during training).

- Forces **redundant representations** — no single neuron can be relied upon
- Equivalent to averaging over 2^n thinned networks
- Typical $p = 0.5$ (hidden layers); $p = 0.1$ after conv layers
- Less effective with Batch Normalization on the same layer



Dropout

Definition (Srivastava et al., 2014)

During training, independently zero out each activation with probability p . At inference, multiply activations by $(1 - p)$ (*inverted dropout* scales during training).

- Forces **redundant representations** — no single neuron can be relied upon
- Equivalent to averaging over 2^n thinned networks
 - Typical $p = 0.5$ (hidden layers), $p = 0.1$ after conv layers
 - Less effective with **Batch Normalization** on the same layer



Dropout

Definition (Srivastava et al., 2014)

During training, independently zero out each activation with probability p . At inference, multiply activations by $(1 - p)$ (*inverted dropout* scales during training).

- Forces **redundant representations** — no single neuron can be relied upon
- Equivalent to averaging over 2^n thinned networks
- Typical $p = 0.5$ (hidden layers); $p = 0.1$ after conv layers
- Less effective with **Batch Normalization** on the same layer



Dropout

Definition (Srivastava et al., 2014)

During training, independently zero out each activation with probability p . At inference, multiply activations by $(1 - p)$ (*inverted dropout* scales during training).

- Forces **redundant representations** — no single neuron can be relied upon
- Equivalent to averaging over 2^n thinned networks
- Typical $p = 0.5$ (hidden layers); $p = 0.1$ after conv layers
- Less effective with **Batch Normalization** on the same layer



Batch Normalization

Definition (Ioffe & Szegedy, 2015)

Normalise activations within a mini-batch, then **scale and shift**:

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

- γ, β are learnable parameters
- Stabilises training: allows larger learning rates
- Reduces sensitivity to initialisation
- Allows for more effective use of variance accumulated during training
- More effective use of variance accumulated during training



Batch Normalization

Definition (Ioffe & Szegedy, 2015)

Normalise activations within a mini-batch, then **scale and shift**:

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

- γ, β are **learnable** parameters
- Stabilises training: allows larger learning rates
- Reduces sensitivity to weight initialisation
- At inference: use running mean/variance accumulated during training
- Alternative: Layer Norm (Transformers), Group Norm (small batches)



Batch Normalization

Definition (Ioffe & Szegedy, 2015)

Normalise activations within a mini-batch, then **scale and shift**:

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

- γ, β are **learnable** parameters
- Stabilises training: allows larger learning rates
- Reduces sensitivity to weight initialisation
- At inference: use running mean/variance accumulated during training
- Alternative: Layer Norm (Transformers), Group Norm (small batches)



Batch Normalization

Definition (Ioffe & Szegedy, 2015)

Normalise activations within a mini-batch, then **scale and shift**:

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

- γ, β are **learnable** parameters
- Stabilises training: allows larger learning rates
- Reduces sensitivity to weight initialisation
- At inference: use running mean/variance accumulated during training
- Alternative: Layer Norm (Transformers), Group Norm (small batches)



Batch Normalization

Definition (Ioffe & Szegedy, 2015)

Normalise activations within a mini-batch, then **scale and shift**:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

- γ, β are **learnable** parameters
- Stabilises training: allows larger learning rates
- Reduces sensitivity to weight initialisation
- At inference: use running mean/variance accumulated during training
- Alternative: Layer Norm (Transformers), Group Norm (small batches)



Batch Normalization

Definition (Ioffe & Szegedy, 2015)

Normalise activations within a mini-batch, then **scale and shift**:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

- γ, β are **learnable** parameters
- Stabilises training: allows larger learning rates
- Reduces sensitivity to weight initialisation
- At inference: use running mean/variance accumulated during training
- Alternative: Layer Norm (Transformers), Group Norm (small batches)



Practical Example: Regularizing a Deep MLP

- **Problem:** overfit a 5-layer MLP on a small dataset
- **Experiment A:** add L2 weight decay $\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}\}$
- **Experiment B:** add Dropout $p \in \{0.2, 0.4, 0.6\}$
- **Experiment C:** insert Batch Normalization before each activation
- **Metric:** validation accuracy and gap to training accuracy
- **Workshop connection:** regularisation setup is required in Challenges 3, 4, 5, and 6



Practical Example: Regularizing a Deep MLP

- **Problem:** overfit a 5-layer MLP on a small dataset
- **Experiment A:** add L2 weight decay $\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}\}$
- **Experiment B:** add Dropout $p \in \{0.2, 0.4, 0.6\}$
- **Experiment C:** insert Batch Normalization before each activation
- **Metric:** validation accuracy and gap to training accuracy
- **Workshop connection:** regularisation setup is required in Challenges 3, 4, 5, and 6



Practical Example: Regularizing a Deep MLP

- **Problem:** overfit a 5-layer MLP on a small dataset
- **Experiment A:** add L2 weight decay $\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}\}$
- **Experiment B:** add Dropout $p \in \{0.2, 0.4, 0.6\}$
- **Experiment C:** insert Batch Normalization before each activation
- **Metric:** validation accuracy and gap to training accuracy
- **Workshop connection:** regularisation setup is required in Challenges 3, 4, 5, and 6



Practical Example: Regularizing a Deep MLP

- **Problem:** overfit a 5-layer MLP on a small dataset
- **Experiment A:** add L2 weight decay $\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}\}$
- **Experiment B:** add Dropout $p \in \{0.2, 0.4, 0.6\}$
- **Experiment C:** insert Batch Normalization before each activation
- **Metric:** validation accuracy and gap to training accuracy
- **Workshop connection:** regularisation setup is required in Challenges 3, 4, 5, and 6



Practical Example: Regularizing a Deep MLP

- **Problem:** overfit a 5-layer MLP on a small dataset
- **Experiment A:** add L2 weight decay $\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}\}$
- **Experiment B:** add Dropout $p \in \{0.2, 0.4, 0.6\}$
- **Experiment C:** insert Batch Normalization before each activation
- **Metric:** validation accuracy and gap to training accuracy
- **Workshop connection:** regularisation setup is required in Challenges 3, 4, 5, and 6



Outline

- 1 Neural Network Fundamentals
- 2 Optimization & Regularization



Thanks!

Questions?



Repo: <https://github.com/EngAndres/ud-public/tree/main/courses/machine-learning>

